

Architectural Analysis: MCP vs. Traditional APIs

Model Context Protocol (MCP) as the AI-Native Integration Layer

Executive Summary: While traditional APIs (REST, GraphQL, gRPC) were built for deterministic, code-to-code communication between software applications, the **Model Context Protocol (MCP)** is an open, AI-native standard designed specifically for autonomous LLM agents. Often described as the 'USB-C for AI', MCP standardizes how language models discover tools, stream live context resources, and execute external actions without requiring bespoke per-application integrations.

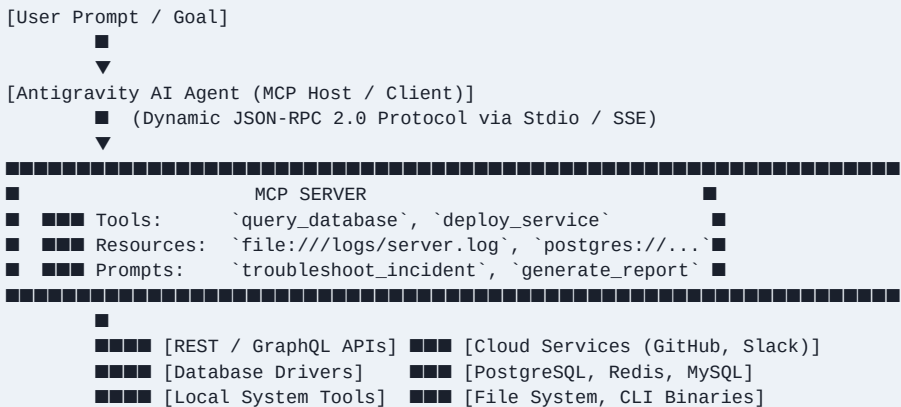
1. Direct Architectural Comparison

Core Dimension	Traditional API (REST / GraphQL / gRPC)	Model Context Protocol (MCP)
Target Consumer	Software Developers & Static Code (requires pre-compiled SDKs or exact endpoints).	AI Agents & LLMs (autonomously reasoned, selected, and parameterized).
Discovery & Schema	Static OpenAPI / Swagger / WSDL docs read by humans during implementation.	Dynamic Runtime Negotiation: The agent automatically queries available tools, schemas, and prompts via JSON-RPC.
Protocol & Transport	HTTP/1.1, HTTP/2, TCP sockets, WebSockets, gRPC.	JSON-RPC 2.0 over Local Stdio (standard I/O) or Remote SSE/HTTP.
Context Primitives	Raw data payloads (JSON, XML, Protocol Buffers).	Three first-class AI primitives: Tools (actions), Resources (context/files), and Prompts (reusable workflows).
Invocation Flow	Deterministic procedural execution: client.createUser(...).	Semantic function calling: Agent determines necessity, extracts arguments from conversation, and handles errors.
Security & Human Gate	Static API keys, OAuth tokens, JWT headers embedded in application code.	Host-level sandboxing, granular permission grants, and User-in-the-Loop approval modals before tool invocation.

2. The Relationship: MCP Does Not Replace APIs — It Wraps Them

A common misconception is that MCP renders APIs obsolete. In reality, **MCP is an abstraction bridge above traditional APIs**:

- **Without MCP:** Every AI application must write custom wrapper code, handle prompt formatting, and map JSON schemas for every distinct REST/GraphQL API.
- **With MCP:** A service authors an MCP Server once. Any MCP-compatible AI client (Google Antigravity, Claude Desktop, Cursor) instantly gains access to its tools and resources without changing application code.



3. The Three Core Primitives of MCP

Unlike APIs which expose generic endpoints, MCP structures agent interactions around three purpose-built primitives:

- **1. Tools (Executable Actions):** Functions with JSON Schema inputs that the agent can autonomously call (e.g. `execute\_query`, `send\_slack\_message`).
- **2. Resources (Contextual Data Streams):** Read-only data attachments (files, schema dumps, live server logs, URI-addressable entities) injected into the prompt context.
- **3. Prompts (Standardized Workflows):** Pre-packaged prompt templates and agent personas published directly by the server (e.g. `security\_audit\_prompt`).

4. MCP Configuration in Antigravity (mcp_config.json)

In Antigravity, MCP servers can be configured globally (~/.gemini/config/mcp_config.json) or at the project level (.agents/mcp_config.json):

```
{
  "mcpServers": {
    "postgres-db": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres", "postgresql://localhost/mydb"],
      "env": { "DEBUG": "true" }
    },
    "github-mcp": {
      "command": "docker",
      "args": ["run", "-i", "--rm", "-e", "GITHUB_PERSONAL_ACCESS_TOKEN", "mcp/github"],
      "env": { "GITHUB_PERSONAL_ACCESS_TOKEN": "ghp_xxxxxxxx" }
    }
  }
}
```

5. Architectural Decision Framework: When to Use What?

- **Use Traditional APIs When:** Building standard software applications, microservices communication, mobile frontends, or high-throughput programmatic data pipelines.
- **Use MCP When:** Connecting AI agents (like Antigravity) to enterprise data, developer tools, database schemas, and proprietary APIs where the LLM needs to decide what tools to invoke.